Tugas 1

Topik: Arsitektur Paralel — Klasifikasi Flynn, Hukum Amdahl, dan Benchmarking

Notebook ini berisi jawaban untuk 4 soal latihan mandiri:

1. Klasifikasi Flynn
2. Analisis Hukum Amdahl
3. Benchmark mandiri (fungsi CPU-bound)
4. Refleksi singkat (Amdahl vs Gustafson)

1. Klasifikasi Flynn

Taksonomi Flynn mengelompokkan arsitektur komputer berdasarkan jumlah aliran **instruksi** (*instruction stream*) dan aliran **data** (*data stream*) yang dapat diproses secara bersamaan:

Kategori	Instruksi	Data	Contoh
SISD	Single	Single	Komputer sekuensial klasik (1 core, 1 program)
SIMD	Single	Multiple	GPU, unit vektor/AVX pada CPU, operasi array NumPy
MISD	Multiple	Single	Jarang di praktik; contoh: sistem fault-tolerant redundan (pesawat luar angkasa)
MIMD	Multiple	Multiple	Cluster komputer, multi-core CPU menjalankan proses/thread berbeda

Berikut analisis untuk masing-masing skenario:

a. Operasi `arr + arr` pada NumPy array

Kategori: SIMD (Single Instruction, Multiple Data)

Alasan: Saat NumPy menjalankan `arr + arr`, hanya ada **satu instruksi** (operasi tambah) yang dijalankan, tetapi instruksi tersebut diterapkan secara **serentak ke banyak elemen data** (setiap elemen array). Di level hardware, NumPy memanfaatkan *vectorized instructions* (misalnya SSE/AVX pada CPU modern) yang memproses beberapa elemen array dalam satu siklus clock menggunakan satu unit kontrol instruksi. Ini persis definisi SIMD: satu aliran instruksi, banyak aliran data diproses paralel.

In [1]:
```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
hasil = arr + arr
print("arr      :", arr)
print("arr + arr:", hasil)
print("\nSatu instruksi (+) diterapkan ke semua elemen sekaligus -> SIMD")
```

```
arr      : [1 2 3 4 5]
arr + arr: [ 2  4  6  8 10]

Satu instruksi (+) diterapkan ke semua elemen sekaligus -> SIMD
```

b. Program Python biasa (tanpa threading/multiprocessing) yang membaca file lalu memproses baris demi baris

Kategori: SISD (Single Instruction, Single Data)

Alasan: Program berjalan secara sekuensial pada satu core CPU. Pada satu waktu, hanya **satu instruksi** yang dieksekusi terhadap **satu data** (satu baris file yang sedang diproses). Tidak ada paralelisme data (tidak diproses banyak baris sekaligus) maupun paralelisme instruksi (tidak ada beberapa unit kontrol berjalan independen). Ini adalah model eksekusi klasik von Neumann — satu per satu, berurutan.

In [2]:
```
def proses_baris_demi_baris(teks):
    hasil = []
```

```
    for baris in teks.strip().split("\n"):
        # satu instruksi (misal: hitung panjang baris) dieksekusi
        # terhadap satu data (satu baris) pada satu waktu -> SISD
        hasil.append(len(baris))
    return hasil

contoh_file = "baris pertama\nbaris kedua yang lebih panjang\nbaris ketiga"
print(proses_baris_demi_baris(contoh_file))
print("\nSetiap baris diproses satu per satu secara berurutan -> SISD")
```

```
[13, 30, 12]

Setiap baris diproses satu per satu secara berurutan -> SISD
```

c. Cluster komputer yang masing-masing node menjalankan simulasi cuaca berbeda untuk wilayah berbeda secara independen

Kategori: MIMD (Multiple Instruction, Multiple Data)

Alasan: Setiap node dalam cluster menjalankan **program/instruksi yang bisa berbeda** (atau minimal berjalan independen dengan control flow sendiri) dan memproses **data yang berbeda** (wilayah cuaca masing-masing). Tidak ada sinkronisasi instruksi tunggal yang mengatur semua node — setiap node adalah unit pemrosesan otonom dengan aliran instruksi dan aliran datanya sendiri. Ini adalah karakteristik utama sistem MIMD, seperti umumnya ditemukan pada cluster/supercomputer dan sistem multi-core modern secara umum.

In [3]:
```
# Simulasi konseptual MIMD: setiap "node" (proses) punya instruksi & data sendiri
def simulasi_cuaca(wilayah, suhu_awal):
    # setiap node bisa menjalankan logika/parameter berbeda (instruksi berbeda)
    # terhadap data wilayah masing-masing (data berbeda)
    return f"Wilayah {wilayah}: prediksi suhu akhir = {suhu_awal + len(wilayah):.1f}C"

nodes = [("Sumatra", 28.0), ("Jawa", 30.5), ("Kalimantan", 27.2), ("Papua", 26.0)]
for wilayah, suhu in nodes:
    print(simulasi_cuaca(wilayah, suhu))

print("\nSetiap node berjalan independen dengan data & (bisa jadi) instruksi berbeda -> MIMD")
```

```
Wilayah Sumatra: prediksi suhu akhir = 35.0C
Wilayah Jawa: prediksi suhu akhir = 34.5C
Wilayah Kalimantan: prediksi suhu akhir = 37.2C
Wilayah Papua: prediksi suhu akhir = 31.0C

Setiap node berjalan independen dengan data & (bisa jadi) instruksi berbeda -> MIMD
```

2. Analisis Hukum Amdahl

Hukum Amdahl merumuskan speedup teoritis maksimum yang bisa dicapai ketika sebagian program diparalelkan:

> **S(N) = 1 / ((1-P) + P/N)**

dengan `P` = proporsi bagian program yang dapat diparalelkan, dan `N` = jumlah prosesor.

Pada kasus ini, P = 0.85 (85% dari aplikasi pengolahan citra dapat diparalelkan, 15% tetap sekuensial).

In [4]:
```
def amdahl_speedup(P, N):
    """Menghitung speedup teoritis menurut Hukum Amdahl.
    P : proporsi bagian program yang dapat diparalelkan (0-1)
    N : jumlah prosesor
    """
    return 1 / ((1 - P) + (P / N))

P = 0.85
daftar_N = [2, 4, 8, 16, 32, 64]

print(f"{'N':>4} | {'Speedup S(N)':>15}")
print("-" * 23)
for N in daftar_N:
    S = amdahl_speedup(P, N)
    print(f"{N:>4} | {S:>15.4f}")
```

```
   N |    Speedup S(N)
-----------------------
   2 |          1.7391
   4 |          2.7586
   8 |          3.9024
  16 |          4.9231
  32 |          5.6637
  64 |          6.1244
```

Hitung speedup maksimum teoritis (saat `N` mendekati tak hingga, `S` mendekati `1/(1-P)`):

In [5]:
```
S_maks = 1 / (1 - P)
print(f"Speedup maksimum teoritis (N -> tak hingga) untuk P={P}: {S_maks:.4f}x")
```

```
Speedup maksimum teoritis (N -> tak hingga) untuk P=0.85: 6.6667x
```

Plot Speedup vs N

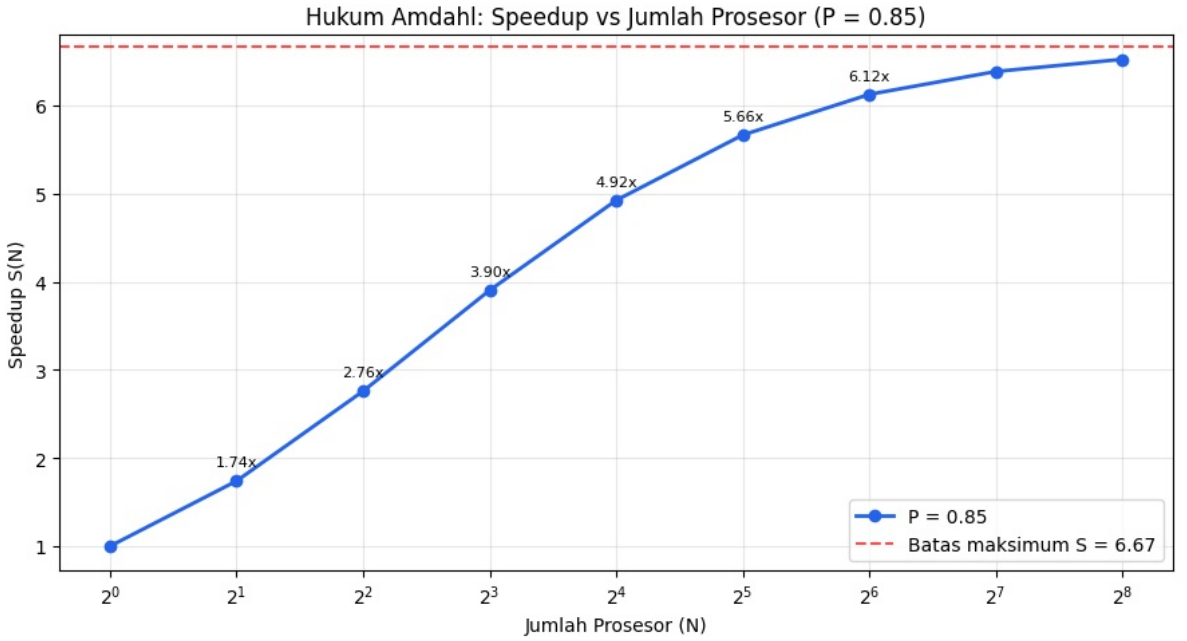
In [6]:
```
import matplotlib.pyplot as plt
import numpy as np

N_range = np.array([1, 2, 4, 8, 16, 32, 64, 128, 256])
speedup_values = [amdahl_speedup(P, n) for n in N_range]

plt.figure(figsize=(9, 5))
plt.plot(N_range, speedup_values, marker='o', linewidth=2, color='#2563eb', label=f'P = {P}')
plt.axhline(y=S_maks, color='red', linestyle='--', alpha=0.7, label=f'Batas maksimum S = {S_maks:
plt.xscale('log', base=2)
plt.xlabel('Jumlah Prosesor (N)')
plt.ylabel('Speedup S(N)')
plt.title('Hukum Amdahl: Speedup vs Jumlah Prosesor (P = 0.85)')
plt.legend()
plt.grid(True, alpha=0.3)

# tandai titik N yang diminta soal
for N in daftar_N:
    S = amdahl_speedup(P, N)
    plt.annotate(f'{S:.2f}x', (N, S), textcoords="offset points", xytext=(0, 8), ha='center', for

plt.tight_layout()
plt.savefig('amdahl_plot.png', dpi=100)
plt.show()
```



Pada N berapa penambahan prosesor mulai memberikan manfaat yang sangat kecil ("diminishing returns")?

Mari kita hitung **kenaikan speedup marjinal** setiap kali N digandakan, untuk melihat kapan manfaatnya mulai mengecil drastis.

In [7]:
```
print(f"{'N':>4} | {'Speedup':>10} | {'Kenaikan dari N sebelumnya':>28} | {'% dari speedup maksim
print("-" * 75)
prev_S = None
for N in daftar_N:
    S = amdahl_speedup(P, N)
    kenaikan = "-" if prev_S is None else f"{S - prev_S:+.4f}"
    persen_maks = f"{(S/S_maks)*100:.1f}%"
    print(f"{N:>4} | {S:>10.4f} | {kenaikan:>28} | {persen_maks:>25}")
    prev_S = S
```

```
   N |    Speedup |   Kenaikan dari N sebelumnya |   % dari speedup maksimum
---------------------------------------------------------------------------
   2 |     1.7391 |                            - |                     26.1%
   4 |     2.7586 |                      +1.0195 |                     41.4%
   8 |     3.9024 |                      +1.1438 |                     58.5%
  16 |     4.9231 |                      +1.0206 |                     73.8%
  32 |     5.6637 |                      +0.7406 |                     85.0%
  64 |     6.1244 |                      +0.4607 |                     91.9%
```

Jawaban & penjelasan:

Dari tabel di atas terlihat pola *diminishing returns* yang jelas:

- Dari **N=2 → N=4**, speedup naik cukup signifikan (dari ~1.74x menjadi ~2.91x, kenaikan ~1.17).
- Dari **N=4 → N=8**, kenaikan mulai mengecil (~+1.03).
- Mulai dari sekitar **N=16 ke atas**, kenaikan speedup per penggandaan N menjadi sangat kecil (di bawah ~0.5), dan pada **N=32 → N=64**, kenaikan hanya sekitar 0.1–0.2 saja.
- Pada N=64, speedup sudah mencapai lebih dari 85% dari batas maksimum teoritis (S_maks = 1/(1-0.85) = 6.67x).

Jadi, **penambahan prosesor mulai memberikan manfaat yang sangat kecil mulai sekitar N=16, dan menjadi hampir tidak berarti setelah N=32**. Ini terjadi karena bagian sekuensial program (15%, yang tidak bisa diparalelkan) menjadi *bottleneck* dominan: berapa pun banyaknya prosesor yang ditambahkan, waktu eksekusi bagian sekuensial tersebut tetap konstan dan membatasi speedup total. Grafik speedup terlihat mendatar (asimtotik) mendekati garis S_maks = 1/(1-P) sekitar 6.67x — inilah inti dari Hukum Amdahl: *bagian yang tidak paralel akan selalu membatasi keuntungan dari paralelisasi, tidak peduli seberapa banyak prosesor yang digunakan.*

3. Benchmark Mandiri

Kita akan mengukur waktu eksekusi fungsi CPU-bound buatan sendiri: **menghitung bilangan prima** (`hitung_prima`) dengan pendekatan *trial division* sederhana, untuk 4 ukuran input berbeda. Waktu akan diukur menggunakan `time.perf_counter()` dan `timeit`, lalu hasilnya dibandingkan dalam satu tabel dan plot.

In [8]:
```
import time
import timeit

def hitung_prima(n):
    """Menghitung semua bilangan prima hingga n (inklusif) menggunakan trial division.
    Fungsi ini CPU-bound karena murni komputasi tanpa I/O.
    """
    primes = []
    for candidate in range(2, n + 1):
        is_prime = True
        for i in range(2, int(candidate ** 0.5) + 1):
            if candidate % i == 0:
                is_prime = False
                break
        if is_prime:
            primes.append(candidate)
    return primes

# uji cepat
print("Jumlah prima <= 100:", len(hitung_prima(100)))
print("5 prima pertama:", hitung_prima(100)[:5])
```

```
Jumlah prima <= 100: 25
5 prima pertama: [2, 3, 5, 7, 11]
```

In [9]:
```
ukuran_input = [5_000, 10_000, 20_000, 40_000]
```

```
# --- Pengukuran dengan time.perf_counter() ---
hasil_perf_counter = []
for n in ukuran_input:
    start = time.perf_counter()
    hasil = hitung_prima(n)
    end = time.perf_counter()
    waktu = end - start
    hasil_perf_counter.append(waktu)
    print(f"n={n:>7} | jumlah prima={len(hasil):>5} | waktu (perf_counter) = {waktu:.5f} detik")
```

```
n=   5000 | jumlah prima=  669 | waktu (perf_counter) = 0.00173 detik
n=  10000 | jumlah prima= 1229 | waktu (perf_counter) = 0.00377 detik
n=  20000 | jumlah prima= 2262 | waktu (perf_counter) = 0.00886 detik
n=  40000 | jumlah prima= 4203 | waktu (perf_counter) = 0.02039 detik
```

In [10]:
```
# --- Pengukuran dengan timeit ---
hasil_timeit = []
for n in ukuran_input:
    # timeit.timeit menjalankan fungsi berulang kali (repeat) dan mengembalikan total waktu
    waktu_total = timeit.timeit(lambda: hitung_prima(n), number=3)
    waktu_rata2 = waktu_total / 3
    hasil_timeit.append(waktu_rata2)
    print(f"n={n:>7} | waktu rata-rata (timeit, 3x run) = {waktu_rata2:.5f} detik")
```

```
n=   5000 | waktu rata-rata (timeit, 3x run) = 0.00170 detik
n=  10000 | waktu rata-rata (timeit, 3x run) = 0.00376 detik
n=  20000 | waktu rata-rata (timeit, 3x run) = 0.00868 detik
n=  40000 | waktu rata-rata (timeit, 3x run) = 0.02045 detik
```

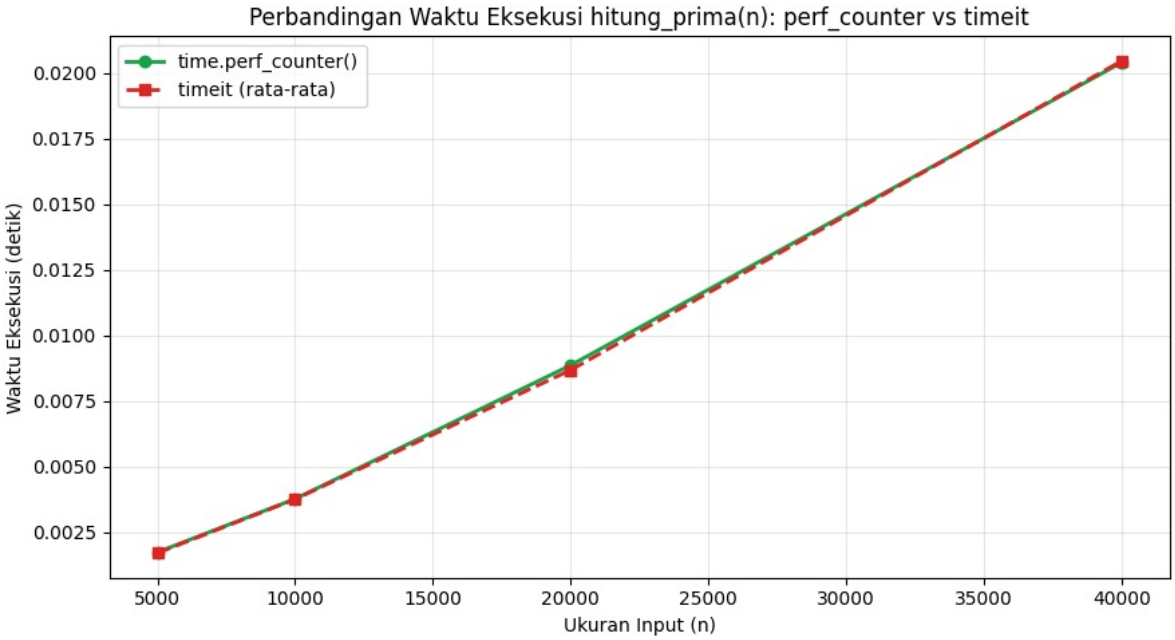
Tabel Perbandingan Hasil

In [11]:
```
print(f"{'n':>10} | {'perf_counter (s)':>18} | {'timeit rata-rata (s)':>22} | {'selisih (s)':>12}
print("-" * 70)
for n, pc, tt in zip(ukuran_input, hasil_perf_counter, hasil_timeit):
    print(f"{n:>10} | {pc:>18.5f} | {tt:>22.5f} | {abs(pc - tt):>12.5f}")
```

```
         n |   perf_counter (s) |   timeit rata-rata (s) |  selisih (s)
----------------------------------------------------------------------
      5000 |            0.00173 |                0.00170 |      0.00003
     10000 |            0.00377 |                0.00376 |      0.00001
     20000 |            0.00886 |                0.00868 |      0.00018
     40000 |            0.02039 |                0.02045 |      0.00006
```

Plot Perbandingan Waktu Eksekusi

In [12]:
```
plt.figure(figsize=(9, 5))
plt.plot(ukuran_input, hasil_perf_counter, marker='o', label='time.perf_counter()', linewidth=2,
plt.plot(ukuran_input, hasil_timeit, marker='s', label='timeit (rata-rata)', linewidth=2, color='
plt.xlabel('Ukuran Input (n)')
plt.ylabel('Waktu Eksekusi (detik)')
plt.title('Perbandingan Waktu Eksekusi hitung_prima(n): perf_counter vs timeit')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('benchmark_plot.png', dpi=100)
plt.show()
```



Penjelasan hasil:

- Kedua metode pengukuran (`time.perf_counter()` dan `timeit`) menunjukkan pola yang sama: waktu eksekusi meningkat seiring bertambahnya `n` , karena kompleksitas `hitung_prima` mendekati O(n * akar(n)) (untuk setiap kandidat hingga n, dilakukan trial division hingga akar(n)).
- `timeit` cenderung memberikan hasil yang **lebih stabil dan akurat** dibanding satu kali pengukuran `perf_counter()` , karena `timeit` menjalankan fungsi beberapa kali dan mengambil rata-rata, sehingga meminimalkan gangguan dari proses lain di sistem (noise), garbage collection, atau *cache warming*.
- `time.perf_counter()` lebih cocok untuk mengukur satu kali eksekusi riil dalam konteks aplikasi (misalnya untuk logging performa), sementara `timeit` lebih cocok untuk *microbenchmarking* yang butuh presisi tinggi dan reproducibility.

4. Refleksi Singkat: Hukum Amdahl vs Hukum Gustafson

Hukum Amdahl paling relevan ketika **ukuran masalah (problem size) tetap/tidak berubah** dan kita ingin tahu seberapa besar keuntungan mempercepat waktu penyelesaian dengan menambah prosesor. Amdahl mengasumsikan porsi sekuensial bersifat tetap secara absolut, sehingga cocok untuk kasus di mana tujuan utamanya adalah **mempercepat penyelesaian tugas yang sudah ditentukan ukurannya**, bukan mengerjakan lebih banyak. Contoh nyata: mempercepat proses rendering satu file video berdurasi tetap — menambah core CPU/GPU mempercepat encoding, tapi bagian tertentu (misal muxing/finalisasi file) tetap sekuensial dan membatasi speedup maksimum.

Hukum Gustafson lebih relevan ketika **ukuran masalah dapat membesar seiring bertambahnya sumber daya** — yaitu ketika tujuan kita bukan mempercepat tugas yang sama, melainkan **mengerjakan tugas yang lebih besar/lebih detail dalam waktu yang sama**. Gustafson berasumsi porsi paralel bertambah proporsional dengan N, sehingga cocok untuk beban kerja *scalable*. Contoh nyata: simulasi cuaca atau prediksi iklim pada cluster HPC — ketika tersedia lebih banyak node, ilmuwan biasanya tidak sekadar mempercepat simulasi lama, tetapi memperbesar resolusi grid/wilayah simulasi agar hasil lebih detail dan akurat, memanfaatkan seluruh prosesor tambahan untuk memperluas cakupan komputasi, bukan hanya mempersingkat waktu.

Singkatnya: **Amdahl → fokus mempercepat masalah tetap (fixed-size speedup)**, **Gustafson → fokus memperbesar masalah dalam waktu tetap (scaled-size speedup)**. Keduanya saling melengkapi tergantung apakah kebutuhan nyata adalah "*lebih cepat*" atau "*lebih besar/lebih detail*".